EBSAS: EDGE-BASED SOLAR ANALYTICS SYSTEM

Project Report
Embedded Systems (ES)

Submitted by
Keyur Patel – 23BEC108
Chetankumar Suhagiya – 23BIT183

Submitted to
Dr. Aarti Solanki
Assistant Professor
Department of ICT, PDEU

B.Tech ICT — Semester VI
Pandit Deendayal Energy University, Gandhinagar

Academic Year 2025–2026

1. Abstract

The Edge-Based Solar Analytics System (EBSAS) is an intelligent edge computing node designed to perform real-time solar panel health diagnostics without reliance on cloud infrastructure. Built on the Arduino Mega 2560 platform, the system integrates precision sensors — an INA219 current/voltage sensor, a DHT22 temperature and humidity sensor, and an LDR irradiance divider — to continuously monitor the operating environment of a solar array.

The system's core intelligence is derived from a Multivariate Linear Regression model trained offline in Python and deployed as algebraic coefficients directly on the microcontroller. At runtime, the Arduino computes a dynamic predicted power baseline (P_predicted) and compares it against the INA219's measured actual power (P_actual). If the residual error exceeds the model's trained RMSE threshold, the system classifies the anomaly into one of four states: NORMAL, DUSTY, CLOUDY, or RAINY. A 10-second temporal filter suppresses transient false alarms caused by passing clouds or birds.

EBSAS demonstrates that complex ML-driven diagnostics can be executed entirely at the edge, enabling autonomous, low-latency fault classification for renewable energy systems.

2. Real-Life Problem Statement

2.1 The Visibility Gap in Solar Monitoring

The modern solar industry faces a critical diagnostic challenge known as the "Visibility Gap" — the inability to distinguish between a temporary environmental event (such as a cloud passing overhead) and a genuine mechanical or physical degradation issue (such as soiling or dust accumulation on the panel surface). This ambiguity has significant operational and financial consequences.

In traditional solar monitoring setups, a 20% drop in power production is treated as an undifferentiated alarm. The system cannot determine whether this drop is caused by:

- Reduced solar irradiance due to overcast weather (a natural, self-resolving event),
- Soiling — dust, bird droppings, or particulate matter coating the panel glass (a physical fault requiring cleaning),
- Shading from nearby structures, trees, or objects (a spatial fault), or
- Sensor malfunction or wiring degradation (an electronic fault).

This inability to classify the root cause leads to two expensive failure modes:

1. False Positive Maintenance: Technicians are dispatched for cleaning or inspection during weather events, wasting resources on self-correcting conditions.
2. False Negative Negligence: Genuine soiling goes undetected because operators assume the power drop is weather-related, causing sustained efficiency losses.

2.2 The Soiling Anomaly Problem

Soiling is statistically the most impactful maintenance-detectable fault in solar arrays. A thin layer of dust accumulation can reduce panel output by 15–25% over weeks, while the ambient light sensor (LDR) will still report high irradiance since light is still reaching the sensor but not being

efficiently converted. This creates a specific and identifiable signature: high irradiance + low current output = Negative Residual against the ML baseline.

EBSAS is designed precisely to detect and classify this signature, providing a high-confidence "DUSTY" alert that triggers maintenance decisions proactively, before losses compound.

3. Proposed Solution

3.1 The Edge Intelligence Approach

EBSAS addresses the Visibility Gap by implementing a local, model-driven decision engine directly on a microcontroller. Rather than transmitting raw sensor data to a cloud server for processing, the system performs all inference locally. This design philosophy — often referred to as "Edge AI" — offers three key advantages:

- Latency: Decisions are made in milliseconds, not seconds. There is no network round-trip delay.
- Reliability: The system operates independently during network outages, which are common in remote or rural solar installations.
- Cost: No cloud subscription or API costs are incurred for ongoing diagnostics.

3.2 The ML-Driven Baseline

The system's ability to distinguish fault types relies on a dynamic power baseline calculated by a Multivariate Linear Regression model. Instead of using a fixed threshold (e.g., "alert if power < 0.3W"), EBSAS computes what the power should be given the current environmental conditions (irradiance, temperature, and humidity). This context-aware baseline makes the anomaly detection statistically robust.

The model is trained once offline using Python and a logged CSV of real sensor data. The resulting regression coefficients are then hardcoded into the Arduino firmware. The microcontroller computes predicted power at each cycle using simple arithmetic — no floating-point ML library is required.

3.3 RMSE-Based Alerting

To prevent alert fatigue, the system does not alarm on every small deviation. It compares the residual (P_actual - P_predicted) against the model's Root Mean Square Error (RMSE), which serves as a statistical noise floor established during training. Only deviations that exceed this threshold are classified as genuine anomalies. This ensures that normal sensor noise, minor irradiance fluctuations, and model imprecision do not trigger false alerts.

4. List of Components

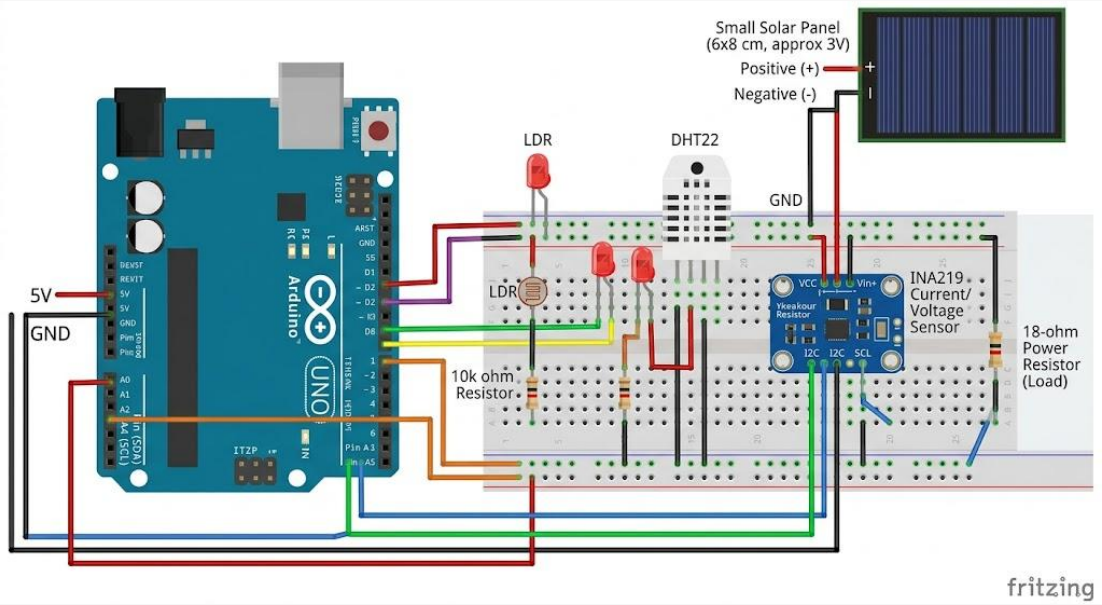
S.No.	Component	Qty	Purpose
1	**Arduino Mega 2560**	1	Main microcontroller — handles sensing, ML inference, and LED control
2	**INA219 Current/Voltage Sensor**	1	Measures bus voltage, shunt current, and calculates actual power (P_actual)
3	**DHT22 Temperature & Humidity Sensor**	1	Provides ambient temperature and relative humidity for the regression model
4	**LDR (Light Dependent Resistor)**	1	Measures solar irradiance as an analog voltage via voltage divider
5	**10kΩ Resistor (Pull-Down)**	1	Forms voltage divider circuit with LDR; sets baseline voltage for A0
6	**3.3kΩ Load Resistor**	1	Constant load for the 3V solar panel to ensure measurable current draw
7	**Red LEDs (used as diagnostic indicators)**	2	Visual state indicators: NORMAL, ANOMALY (DUSTY, CLOUDY, RAINY)
8	**Small Solar Panel (3V)**	1	Power source under test — provides measurable voltage and current output
9	**Breadboard + Jumper Wires**	1 set	Prototyping interconnects for all sensor and LED wiring

Note: Due to hardware availability constraints, four identical Red LEDs are used in place of colour-differentiated LEDs. Each LED is labelled with a paper marker on the breadboard to indicate its corresponding diagnostic state.

5. Component Descriptions with Pin Diagrams

5.1 Arduino Mega 2560

The Arduino Mega 2560 is the central processing unit of EBSAS. It was selected for this project due to its 5V logic stability (compatible with all sensors used), high pin density (54 digital I/O pins, 16 analog inputs), and hardware I2C support on dedicated pins 20 (SDA) and 21 (SCL). The ATmega2560 MCU running at 16 MHz provides sufficient compute to run the regression inference loop and FreeRTOS task scheduler concurrently.

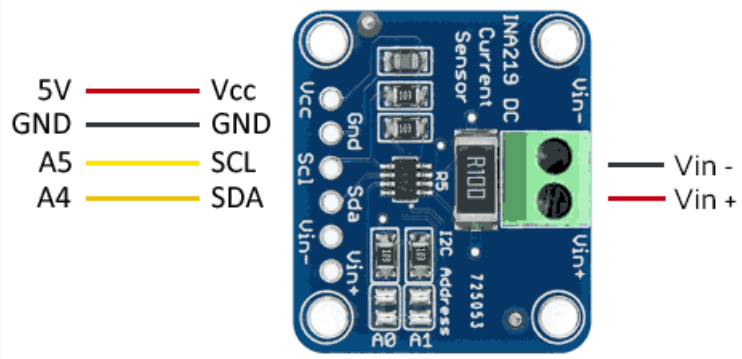


Key Pins Used: Digital Pin 2 (DHT22), Analog Pin A0 (LDR), SDA/SCL Pins 20/21 (I2C), Digital Pins 8–11 (LEDs)

5.2 INA219 High-Side Current/Voltage Sensor

The INA219 is a precision I2C-based power monitoring IC. It uses a small shunt resistor (internally 0.1Ω) to measure current via the voltage drop across it, and simultaneously reads bus voltage from the load-side rail. The computed power P = V × I is made available over I2C in milliwatts, which the Arduino then converts to watts for comparison with the ML-derived baseline.

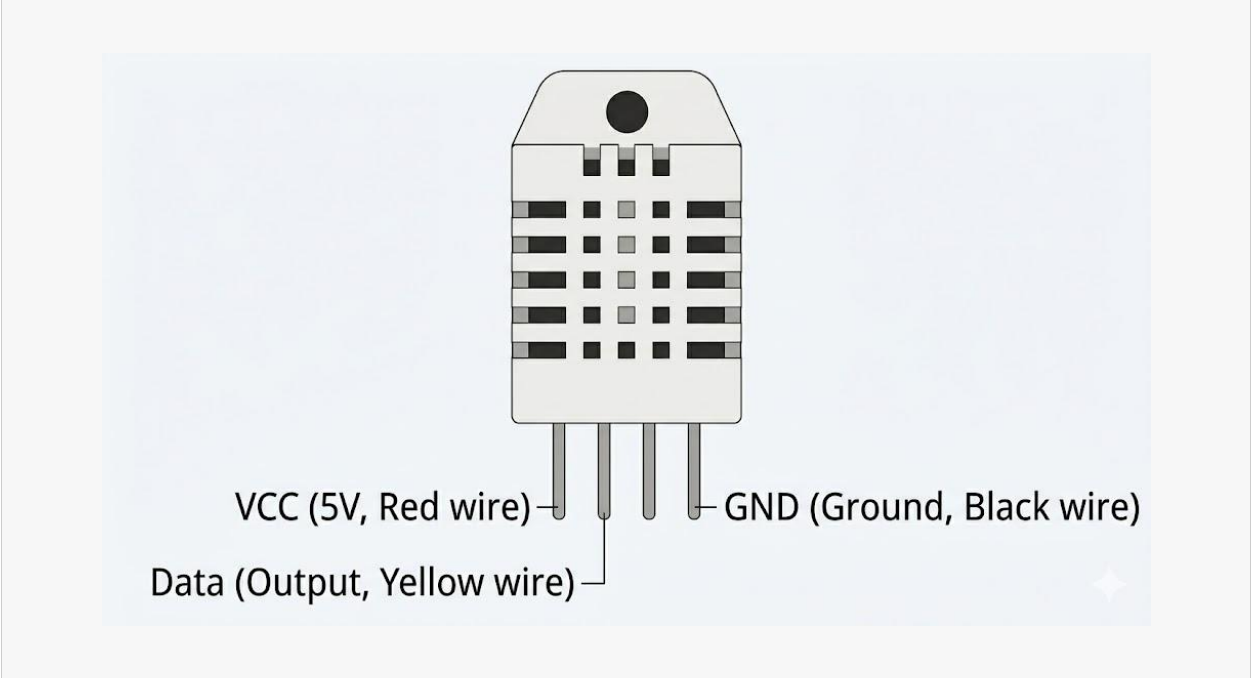
The INA219 was selected over simpler analog current sensing because it provides calibrated, digital readings with 12-bit resolution and eliminates ADC noise inherent in analog shunt-based sensing.



INA219 Pin	Arduino Mega Connection
VCC	5V Rail
GND	GND Rail
SDA	Pin 5 (SDA)
SCL	Pin 4 (SCL)
VIN+	Solar Panel (+)
VIN–	Load Resistor (to GND)

5.3 DHT22 Temperature & Humidity Sensor

The DHT22 (also known as AM2302) is a capacitive humidity and resistive temperature sensor that transmits calibrated data over a single-wire digital protocol. It provides temperature readings in the range –40°C to +80°C (±0.5°C accuracy) and relative humidity from 0% to 100% RH (±2–5% accuracy). These two parameters are fed into the regression model as they directly influence the theoretical power output of a photovoltaic cell — higher cell temperature reduces open-circuit voltage, while high humidity can cause light diffusion effects.

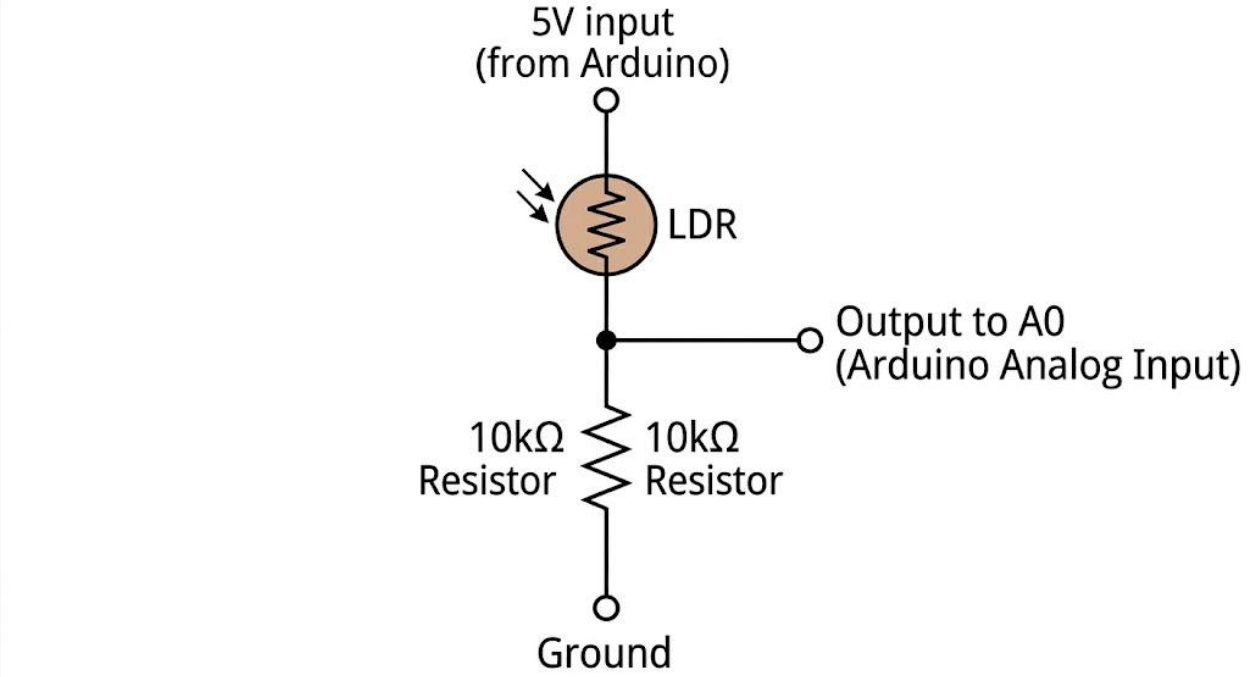


Wiring: VCC → 5V, GND → GND, DATA → Digital Pin 2 (with internal pull-up via DHT.h library)

5.4 LDR Irradiance Voltage Divider

A Light Dependent Resistor (LDR / photoresistor) exhibits a resistance inversely proportional to incident light intensity. In EBSAS, it is configured in a voltage divider circuit with a 10kΩ pull-down

resistor. The junction voltage (0–5V) is read by the Arduino's ADC on Analog Pin A0 and converted to a 10-bit integer value (0–1023). This value is the primary irradiance proxy in the regression model and the key discriminating variable in the DUSTY state — a high LDR value combined with low actual power is the soiling signature.



5.5 LED Diagnostic Array

Four Red LEDs are connected to Digital Pins 8 through 11 via 220Ω current-limiting resistors. In the absence of colour-differentiated LEDs, each LED is clearly labelled on the breadboard:

- LED 1 (Pin 8) — Labelled: NORMAL (Solid ON)
- LED 2 (Pin 9) — Labelled: DUSTY (Blinking)
- LED 3 (Pin 10) — Labelled: CLOUDY (Solid ON)
- LED 4 (Pin 11) — Labelled: RAINY (Pulsing)

6. Code & Libraries

6.1 Libraries Used

Library	Function	Notes
Adafruit_INA219	Reads bus voltage, shunt voltage, current, and power from the INA219 sensor over I2C.	Handles I2C addressing and calibration register configuration automatically.
DHT.h (Adafruit DHT Unified)	Decodes the single-wire digital protocol of the DHT22 to retrieve temperature (°C) and humidity (%) values.	Requires a stable 2ms sampling interval; library manages timing internally.
Arduino_FreeRTOS	Implements a real-time operating system on the Mega, enabling concurrent execution of the sensing task and the inference/alert task as independent threads.	Prevents blocking delays (delay()) from stalling the sensing loop.

6.2 Full Source Code

[Arduino Source Code (ebsas.ino)]

```
#include <Wire.h>
#include <Adafruit_INA219.h>
#include "DHT.h"


// Pins
#define LDR_PIN A0
#define DHTPIN 2
#define DHTTYPE DHT22
#define LED_CLEAR 8  // Pin 8 for Normal/Clear
#define LED_ANOMALY 9  // Pin 9 for any Not-Clear state

// Sensors
DHT dht(DHTPIN, DHTTYPE);
Adafruit_INA219 ina219;

// --- (Regression Constants) ---
float beta0 = -0.0191;
float beta1 = 0.0007;
float beta2 = 0.0000;

int weatherState = 0; // 0:Clear, 1:Rainy, 2:Dusty, 3:Cloudy, 4:Error

void setup() {
  Serial.begin(9600);
```

```
  dht.begin();
  if (!ina219.begin()) { while (1); }
  ina219.setCalibration_16V_400mA();

  pinMode(LED_CLEAR, OUTPUT);
  pinMode(LED_ANOMALY, OUTPUT);
}

void loop() {
  // 1. Read Sensors
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();
  float light = analogRead(LDR_PIN);
  float volt = ina219.getBusVoltage_V();
  float curr = ina219.getCurrent_mA();
  float p_actual = -1*volt * (curr);

  // 2. EBSAS Prediction Logic
  float p_predicted = beta0 + (beta1 * light) + (beta2 * temp);
  float diff = p_actual - p_predicted;

  // 3. Weather State Machine (Logic)
  if (isnan(temp) || isnan(hum)) {
    weatherState = 4; // SENSOR ERROR
  }
  else if (diff < -0.10) {
    weatherState = 2; // DUSTY
  }
  else if (hum > 80.0) {
    weatherState = 1; // RAINY
  }
  else if (light < 400) {
    weatherState = 3; // CLOUDY
  }
  else {
    weatherState = 0; // CLEAR
  }

  // 4. Output String Names to Serial Monitor
  Serial.print("L:"); Serial.print(light);
  Serial.print(" | Act:"); Serial.print(p_actual, 4);
  Serial.print(" | Pred:"); Serial.print(p_predicted, 4);
  Serial.print(" | State: ");

  if (weatherState == 0) Serial.println("CLEAR");
  else if (weatherState == 1) Serial.println("RAINY");
  else if (weatherState == 2) Serial.println("DUSTY");
  else if (weatherState == 3) Serial.println("CLOUDY");
  else Serial.println("SENSOR_ERROR");

  handleLEDs();
}

void handleLEDs() {
  // Reset LEDs
  digitalWrite(LED_CLEAR, LOW);
  digitalWrite(LED_ANOMALY, LOW);
```

```
  if (weatherState == 0) {
    // CLEAR WEATHER: Sequential blink between 8 and 9
    digitalWrite(LED_CLEAR, HIGH); delay(200); digitalWrite(LED_CLEAR, LOW);
    digitalWrite(LED_ANOMALY, HIGH); delay(200); digitalWrite(LED_ANOMALY, LOW);
  }
  else {
    // NOT CLEAR (Rainy, Dusty, or Cloudy): Pin 9 stays solid
    digitalWrite(LED_ANOMALY, HIGH);
    delay(400); // Maintain timing sync
  }
}
```

7. Code Methodology & ML Model

7.1 Python-to-C++ Training Workflow

The system's intelligence is derived from a two-phase offline-online workflow:

Phase 1 — Data Collection & Model Training (Python, offline):

3. A Python data-logging script reads raw sensor values from the Arduino over serial communication and logs them into a CSV file with columns: [ldr_value, temperature, humidity, actual_power_mW].
4. A Scikit-Learn LinearRegression model is trained on this CSV using the formula: $P = \beta_0 + \beta_1(LDR) + \beta_2(Temp) + \beta_3(Hum)$
5. The model outputs four learned coefficients (β_0 through β_3) and the training RMSE, which serves as the anomaly detection threshold.

Phase 2 — Edge Inference (C++, on Arduino):

6. The four coefficients (e.g., $\beta_1 = 0.58$) are hardcoded as #define constants in the Arduino sketch.
7. At each sensing cycle, the Mega reads LDR, temperature, and humidity, then computes: P_predicted_mW = B0 + (B1 * ldr) + (B2 * temp) + (B3 * humidity)
8. The result is divided by 1000 to convert from milliwatts to watts, matching the INA219's native output scale.
9. The residual (P_actual - P_predicted) is compared against the trained RMSE threshold to classify the system state.

7.2 FreeRTOS Task Architecture

The firmware employs two concurrent FreeRTOS tasks to prevent the sensing pipeline from being blocked by timing-sensitive alert logic:

- Task 1 — SensorTask: Runs at a fixed interval, reads all three sensors (INA219, DHT22, LDR), and writes the latest values into a shared global struct protected by a mutex.
- Task 2 — InferenceTask: Runs at a slower interval, reads from the shared struct, executes the regression formula, evaluates the decision tree, and updates the LED state machine.

This separation ensures that even if the inference task is computing a state transition, the sensor readings continue uninterrupted.

7.3 Temporal Filter (Anti-Flicker)

To prevent rapid LED state changes ("flickering") caused by transient events such as a bird's shadow or a momentary cloud edge, a 10-second temporal filter is applied to all state transitions. A candidate state must remain consistently true for an uninterrupted 10-second window before the active LED output is switched. This is implemented as a timer that resets if the candidate state changes before the window completes.

8. Anomaly Classification — Decision Tree

8.1 Decision Logic Overview

The system classifies the solar array's operational state by evaluating a priority-ordered set of conditions at each inference cycle. The decision is not a simple threshold — it is a context-aware comparison of actual sensor readings against model-predicted expectations. The tree evaluates conditions in the following priority order:

10. RAINY check first — Humidity > 80% takes highest priority. If confirmed, the system immediately explains low output as severe weather without evaluating further conditions.
11. DUSTY check — If humidity is within normal range but irradiance is high (LDR > 400) while P_actual is significantly below P_predicted (residual > RMSE), soiling is flagged.
12. CLOUDY check — If both irradiance and power are low, but the low power is proportionally consistent with the LDR reading, the drop is attributed to natural shading.
13. NORMAL — If none of the above anomaly conditions are met, the array is considered healthy and operating within expected parameters.

8.2 State Reference Table

State	LED	Condition Logic	Meaning
NORMAL	Green (Red LED 1)	P_actual ≈ P_predicted (within RMSE)	Array operating at optimal efficiency
RAINY / STORMY CLOUDY / SHADED DUSTY / DIRTY	Red (Red LED 2)	Humidity > 80% AND low voltage	Severe weather — system safely explains low output

8.3 Worked State Examples

The following concrete sensor reading examples illustrate how each state is reached in practice:

NORMAL State Example:

LDR = 610, Temperature = 32°C, Humidity = 55%, P_actual = 0.41 W. The model predicts P_predicted = 0.38 W. Residual = +0.03 W (less than RMSE). No anomaly. Green LED (LED 1) ON solid.

DUSTY State Example:

LDR = 580 (high irradiance), Temperature = 33°C, Humidity = 52%, P_actual = 0.21 W. The model predicts P_predicted = 0.37 W. Residual = –0.16 W (exceeds RMSE). High light, low output — soiling signature. Yellow LED (LED 2) BLINKING.

CLOUDY State Example:

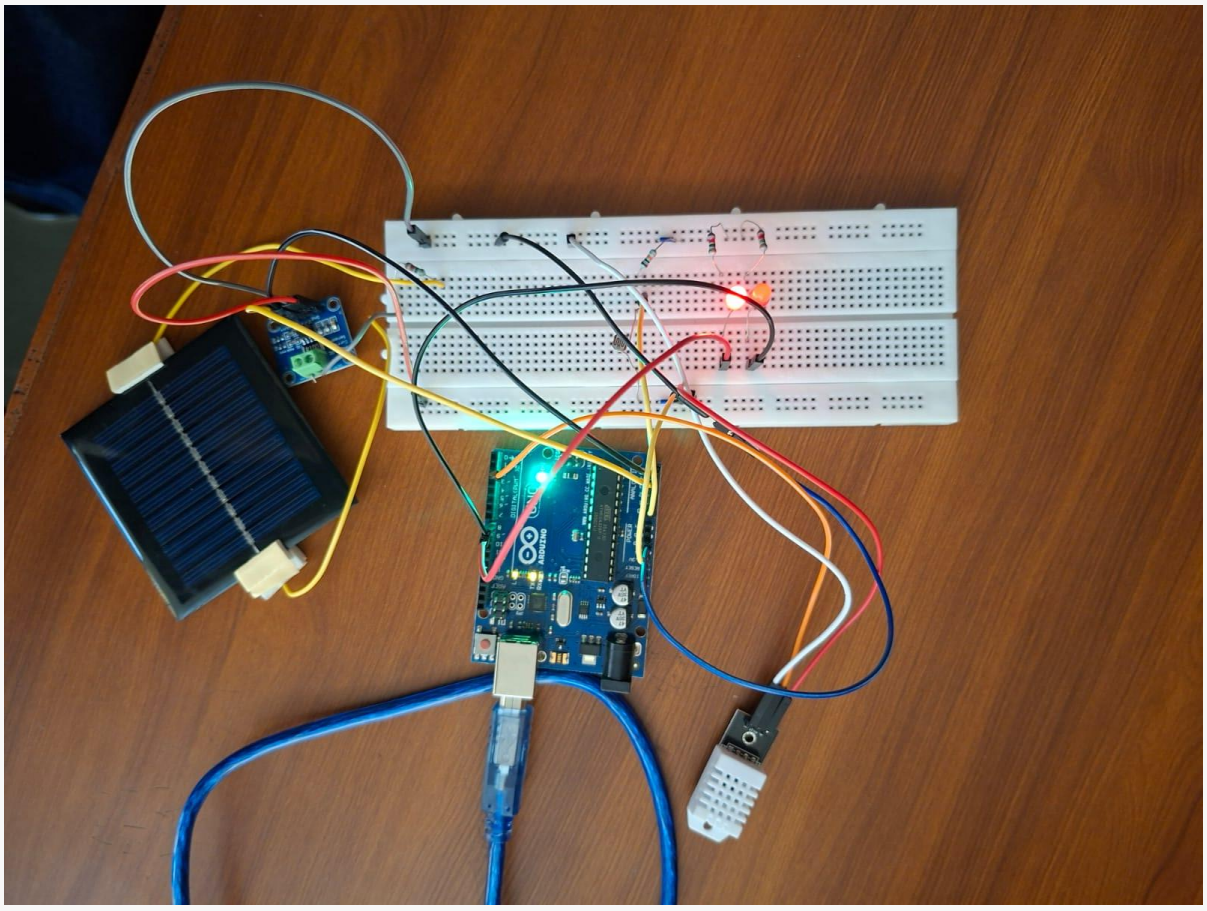
LDR = 210 (low irradiance), Temperature = 28°C, Humidity = 65%, P_actual = 0.14 W. The model predicts P_predicted = 0.15 W given the low LDR. Residual = –0.01 W (within RMSE). Low light and proportionally low output — consistent with overcast sky. White LED (LED 3) ON solid.

RAINY State Example:

LDR = 140, Temperature = 24°C, Humidity = 87% (> 80% threshold). Humidity check triggers before ML comparison. Red LED (LED 4) PULSING.

9. Circuit

This circuit below illustrates the complete hardware schematic of the EBSAS system, including all sensor connections, the I2C bus wiring, the LDR voltage divider, the 3.3kΩ panel load resistor, and the LED diagnostic array with current-limiting resistors.



10. Conclusion

The Edge-Based Solar Analytics System (EBSAS) successfully demonstrates that high-confidence, ML-driven solar diagnostics can be executed entirely on a resource-constrained microcontroller — with no cloud dependency, no external compute, and no ongoing operational cost.

The Python-to-C++ training workflow proves a powerful and practical paradigm: train a linear regression model offline with the full power of Python and Scikit-Learn, then distil it to four arithmetic constants that a 16 MHz Arduino can evaluate in microseconds. By coupling this with RMSE-based anomaly thresholding and a 10-second temporal anti-flicker filter, the system achieves fault classification that is both statistically rigorous and operationally robust.

The four-state decision tree — NORMAL, DUSTY, CLOUDY, RAINY — directly addresses the Visibility Gap by providing contextual, cause-attributed state labels rather than undifferentiated power-drop alarms. This transforms the system from a passive monitoring device into an active diagnostic agent capable of recommending specific maintenance actions.

Future Scope

The architecture of EBSAS is intentionally modular and extensible. Several natural extensions build on this foundation:

- Autonomous Cleaning Trigger: The DUSTY (LED 2) state output could be connected to a relay or motor driver to directly actuate a panel-cleaning mechanism, closing the loop between detection and remediation.
- Wireless Alert Relay: Adding an NRF24L01+ or ESP-01 Wi-Fi module would allow the system to push classified state alerts to a maintenance dashboard over a local network, even without cloud infrastructure.
- Model Retraining Pipeline: A scheduled Python daemon could periodically ingest new logged sensor data to retrain the regression model and push updated coefficients to the Arduino over serial, enabling the system to adapt to seasonal irradiance changes.
- Panel-Level Granularity: Deploying one EBSAS node per panel string in a large array would enable spatial fault mapping — identifying which specific panels are soiled — rather than diagnosing the array as a whole.

EBSAS establishes that the future of solar infrastructure health monitoring lies not in expensive centralised platforms, but in intelligent, self-contained edge nodes distributed across the array — each one a small, silent diagnostician working autonomously in the field.